

AI Algorithm

LLM Optimization



Marcel YAMMINE

François ZAPLETAL

Anthony YANG

DIA 6

2025/2026

Table des matières

Introduction:	2
Why is LLM Optimization Critical?	2
The Challenges We Face	2
Our Approach	3
Application of AI Techniques in your Chosen Area:.....	3
Quantization	3
Low-Rank Factorization	3
Pruning	4
Knowledge Distillation	5
Implementation of a Selected Technique:	6
Description of the Sub-Problem	6
Justification of the Approach	8
Empirical Evaluation:	8
1. Experimental Setup.....	9
2. Results	9
3. Assumptions, Limitations, and Trade-Offs	10
Conclusion:	10
Main trends in AI-based LLM optimization.....	10
What we learned from the survey and our implementation	11
Future research directions we recommend.....	12

Introduction:

Why is LLM Optimization Critical?

Large Language Models have become incredibly powerful, but there's a problem: they're too big to actually use. GPT-3 needs 350GB just to load into memory, and newer models are even larger. This creates a real deployment issue the best models are often impossible to run outside of expensive cloud infrastructure.

The costs pile up quickly. Running these models at scale can cost thousands per day in compute alone. For most companies and research groups, this just isn't sustainable. Beyond the financial burden, there's also an environmental cost, training and running these models generates significant CO2 emissions.

Then there's the accessibility problem. If only big tech companies can afford to run these models, we're creating a massive barrier to entry. Researchers in smaller institutions, startups, and developing countries get left behind. And forget about running these models on phones or edge devices, they're simply too large.

This is why optimization matters. We need to find ways to shrink these models down while keeping them useful.

The Challenges We Face

Making LLMs smaller isn't straightforward. Several things make this particularly difficult:

Different layers behave differently: some parts of the model are really sensitive, compress them even a little and performance decreases. Other parts have lots of redundancy and can handle aggressive compression just fine. The problem is figuring out which is which. If you compress everything uniformly, you either don't compress enough (missing opportunities) or compress too much (breaking the model).

Most compression decisions aren't differentiable: standard neural network training uses gradient descent, but many compression choices are discrete like deciding how many blocks to split a matrix into, or whether to use 4-bit or 8-bit quantization. You can't just backpropagate through these decisions. This forces us to use different optimization approaches like evolutionary algorithms, which are slower and less predictable.

There's always a tradeoff: better compression usually means worse accuracy. And better depends on what you need sometimes you care about memory, sometimes speed, sometimes power consumption. These don't always align. For example, making a model sparse can reduce memory but might not actually speed things up on standard hardware.

It's hard to know what is broken: standard benchmarks like accuracy on test sets don't tell the whole story. Modern LLMs have these emergent abilities things like chain-of-thought reasoning that only appear at scale. Compress too much and you might lose these without realizing it until you deploy.

Our Approach

We tried to address these issues by combining three ideas: first, we analyze which layers are least sensitive to compression (so we know where to compress aggressively). Second, we use block-wise SVD instead of compressing whole matrices at once (this lets us exploit local structure). Third, we use a genetic algorithm to search for good compression configurations (since we can't use gradient descent for discrete choices).

The key insight was that compression needs to be adaptive. Different parts of the model need different treatment, and you need automated search to figure out the right configuration.

Application of AI Techniques in your Chosen Area:

Quantization

1. Trade-offs

Quantization reduces the precision of model weights and activations (for example, FP16 $\rightarrow$ 8-bit or 4-bit). Its main advantage is efficiency: memory footprint shrinks proportionally to bit-width, and integer arithmetic speeds up inference. Methods such as GPTQ, AWQ, and QLoRA show that even 4-bit quantization can preserve accuracy for many LLMs.

The trade-off is increased quantization noise. Some layers, especially attention and projection layers, are highly sensitive to precision loss. The lower the bit-width, the more calibration is needed to avoid instability. Quantization also works better for inference than training, since backpropagation amplifies rounding errors.

2. Critical Perspective

Quantization is well suited for deployment on resource-constrained hardware, where memory limits dominate. It is easy to integrate in existing models, requires no architectural changes, and provides immediate speed improvements.

Its main limitations are stability at very low precision and limited benefits for compute-bound settings. Quantization does not reduce the number of operations, only their cost. From our perspective, quantization is highly practical but rarely sufficient alone, especially for fine-tuning or domain adaptation where precision is important.

2. Future Directions

Future work is moving toward adaptive bit-width selection, where each layer (or channel) receives its own precision level. Data-aware quantization, which adjusts precision based on activation distributions, is also promising. Another direction is using optimization algorithms (such as, RL or evolutionary search) to discover optimal quantization schemes automatically. Finally, stabilizing low-bit training remains an open challenge.

Low-Rank Factorization

1. Trade-offs

Low-rank factorization assumes that LLM weight matrices contain redundancy and can be approximated using smaller matrices (like with SVD or LoRA). This approach often preserves accuracy better than aggressive quantization because the approximation remains in high precision.

The trade-off is that factorization does not always reduce activation memory, and in some cases adds extra matrix multiplications. Another challenge is choosing the correct rank for each layer. The *SVD-LLM V2 paper* shows that uniform truncation is suboptimal; different layers require different compression ratios to avoid accuracy loss.

2. Critical Perspective

Low-rank factorization is very suitable for fine-tuning and scenarios where preserving quality is essential. LoRA shows that adapting only low-rank updates dramatically reduces the number of trainable parameters while maintaining strong performance.

However, factorization requires more tuning than quantization. Some layers simply do not admit a low-rank structure without degrading performance, and selecting truncation levels remains a nontrivial optimization problem. In our view, low-rank methods are powerful but less “plug-and-play” than quantization, and they benefit from complementary techniques rather than working alone.

3. Future Directions

A key direction is layer-wise adaptive rank selection, as demonstrated by SVD-LLM V2. Another promising path is combining low-rank factorization with quantization—high precision for the low-rank components, low precision for the rest—to balance stability and efficiency. Automated search methods could also be used to determine optimal ranks across layers, turning truncation into a structured optimization task. Lastly, exploring low-rank structures in attention mechanisms may unlock further gains.

Pruning

1. Trade-offs

Pruning targets structural redundancy in LLMs by removing weights or whole components that contribute little to model prediction. Transformer models contain billions of parameters and large feed-forward blocks, resulting in high static memory use, while inference additionally requires variable KV-cache memory that grows with sequence length and batch size. Pruning reduces model size by evaluating parameter importance (saliency) either through loss sensitivity (gradient/Hessian metrics) or through layer-output preservation using reconstruction approximations, resulting in smaller models that maintain most of their accuracy while reducing memory footprint and latency. Its main trade-off is that not all pruning provides real speedups: removing weights does not always reduce computation if the remaining model is still dense, making some pruning highly effective for compression but less impactful in runtime.

2. Critical Perspective

Pruning is well suited for deployment when model size or memory limits are the primary issues. It can be applied post-training and does not require architectural redesigns, making it attractive in practical workflows. However, the benefit depends strongly on pruning granularity. Unstructured sparsity maximizes theoretical compression but often fails to accelerate inference on current hardware, whereas structured pruning offers real speed improvements but at lower sparsity levels. A recent evolution is dynamic pruning, where pruning decisions adapt during inference to memory availability, overcoming limitations of static sparsity under volatile KV-cache requirements. From our perspective, pruning is powerful but uneven: highly useful for memory efficiency, conditionally useful for speed, and increasingly critical when inference constraints change over time.

3. Future Directions

Future research is moving toward hybrid hardware-aware pruning, where models choose between structured and unstructured removal depending on accelerator characteristics. Dynamic inference-time pruning, as in RAP, suggests that pruning will evolve into adaptive systems that respond to sequence length, batch size, or latency constraints. Further progress may involve optimization algorithms (e.g., RL or evolutionary search) to learn optimal pruning policies automatically. Stabilizing performance under high sparsity (especially for attention and FFN blocks) remains a major challenge, and future pruning must account for this sensitivity while ensuring real-world deploy ability.

Knowledge Distillation

1. Trade-offs

Knowledge Distillation (KD) is a technique where a smaller, efficient student model is trained to emulate the behavior of a larger, high-performing teacher LLM. This is achieved by having the student learn from the teacher's soft outputs (probability distributions), which contain more informative "dark knowledge" than simple hard labels. The primary benefit of KD is the substantial reduction in model size and a corresponding increase in inference speed, as it allows for the deployment of a fundamentally simpler architecture. A secondary advantage is that the student model may sometimes generalize better due to learning from the teacher's smooth distributions.

However, the core trade-off is the significant computational cost and time required for the distillation phase itself, making the initial training cycle potentially slower than other compression methods like Post-Training Quantization (PTQ). Crucially, the student's final performance is inherently limited by the teacher's quality, carrying the risk of a capacity transfer gap, where the smaller model cannot replicate the complex, emergent reasoning capabilities of the teacher, especially in specialized or highly complex tasks.

2. Critical Perspective

KD is widely regarded as the most powerful compression method for creating task-specific Small Language Models (SLMs) designed for resource constrained environments. Unlike pruning or quantization, KD allows for a complete architectural redesign. From a critical

standpoint, the primary hurdle in applying KD to modern LLMs is the necessity to transfer deep cognitive skills such as Chain-of-Thought or in context learning instead of merely matching final predictions. Traditional KD methods often fail to capture this nuanced, higher order knowledge, leading to models that perform well on simple tasks but fail on complex generative reasoning. Another practical limitation is the reliance on obtaining or generating high-quality, task-specific data to effectively facilitate knowledge transfer, a major initial hurdle that contrasts with low-resource deployment goals. This is increasingly addressed by leveraging the teacher model itself to generate synthetic training data.

3. Future Directions

Future research in Knowledge Distillation is focused on improving the quality and completeness of knowledge transfer from teacher to student:

- **Rationale-Based Distillation:** This involves training the student model to directly mimic the intermediate reasoning steps or rationales generated by the teacher, which helps the student inherit the teacher's logical abilities and improves generalization.
- **Multi-Teacher and Multi-Tier Distillation:** Using multiple specialized teachers or cascading the distillation process (large $\rightarrow$ medium $\rightarrow$ small) is being explored to effectively bridge the large capacity gap between massive teachers and very small students, making the training process more stable.
- **Integration with Data Distillation (DD):** A dual-focus approach involves combining KD with Data Distillation, which condenses large original datasets into compact, synthetic, information rich training sets, addressing both model and data efficiency.
- **On Policy Distillation:** This promising area applies Reinforcement Learning (RL) principles, where the teacher provides a fine grained, per token reward signal to guide the student model's training, particularly effective for instruction following capabilities.

Implementation of a Selected Technique:

Description of the Sub-Problem

The main issue we address is the excess of parameters in Large Language Models. Transformers tend to use far more weights than necessary for a specific downstream task, which makes them heavy to store and expensive to run. Many compression methods simply apply the same reduction everywhere in the network, but this is rarely optimal: some layers are extremely sensitive and cannot be touched, while others contain large amounts of redundancy and tolerate aggressive compression.

In our case, we treat the problem as a multi-objective optimisation task. Using a DistilBERT model fine-tuned on SST-2, we seek an automatic way to decompose its Linear layers so that the overall model becomes lighter while preserving accuracy. Instead of fixing a single rank for every layer, we want the compression pattern to adapt itself depending on how each layer behaves.

To achieve this, we designed a three-phase pipeline combining sensitivity analysis, block-wise low-rank factorisation, and evolutionary optimisation via a genetic algorithm.

Phase 1: Sensitivity-Based Layer Ranking (SBLR)

Before deciding how to compress, we need to know where compression is even allowed. The intuition is simple: a layer that is robust to noise is also more likely to tolerate compression.

For each Linear layer, we add a small Gaussian perturbation with amplitude $\varepsilon = 0.1$

only to that layer's weight matrix, then measure the drop in accuracy on a validation split. This gives a sensitivity score per layer.

We then rank all layers and select the eight least sensitive ones for compression. These are mainly feed-forward layers and some attention projections. The fragile layers are left untouched. This step reflects an important idea also found in the literature (e.g. SVD-LLM V2): knowing *where* to compress is as important as knowing *how much* to compress.

Phase 2: Block-wise Singular Value Decomposition (Block-SVD)

Once the target layers are identified, we move to the design of the compression itself. A global SVD of a weight matrix $W = U\Sigma V^T$ followed by truncation gives a classical low-rank approximation, but it assumes that the entire matrix behaves uniformly.

Instead, we split each matrix into a grid of N_b independent blocks and apply SVD to each block B_i so that $B_i \approx U_i \Sigma_i V_i^T$.

Each block receives its own truncation rank r_i , allowing strong compression in redundant regions and higher fidelity where the matrix carries important structure.

At first, we reconstructed a dense matrix from the truncated SVD, which produced no real memory gain. The fix was architectural: we created a custom PyTorch module, *BlockLowRankLinear*, which stores all U_i , Σ_i , and V_i factors directly and performs the forward pass without ever rebuilding W .

A block with dimensions $m \times n$ now requires roughly $r(m + n)$ parameters instead of mn , which leads to an actual reduction in model size.

Phase 3: Evolutionary Optimisation with a Genetic Algorithm

The final step is selecting the best configuration of blocks and ranks for each compressible layer. The choices are discrete: the number of blocks may be 4, 8, or 16, and each block rank r_i is an integer within a fixed range. Combining these across several layers creates a search space too large to explore manually, and unsuitable for gradient descent.

We encode each candidate solution as a chromosome, represented as a dictionary mapping each layer to its number of blocks and the list of ranks assigned to those blocks.

To evaluate candidates, we use a normalised fitness function that balances accuracy with size:

$$F = \frac{\text{accuracy}}{\text{baseline accuracy}} - \lambda \times \frac{\text{model size}}{\text{baseline size}}$$

With $\lambda = 0.5$.

This rewards models that maintain good accuracy while significantly reducing memory.

We use standard genetic operators: tournament selection to choose parents, one-point crossover to exchange layer configurations, random mutation to alter block counts or ranks, and elitism to preserve the strongest solutions across generations. Because each candidate requires rebuilding a model and running inference, we keep the population size moderate, but the algorithm still converges toward stable, meaningful configurations.

Justification of the Approach

This three-stage pipeline is not the only way to compress a model, but there are clear reasons why it makes sense in our setting. First, it gives us fine-grained control over where and how we compress. Unlike pruning, which either removes individual weights (unstructured sparsity) or whole units (structured pruning), block-wise low-rank factorisation lets us keep more capacity in the parts of a matrix that matter and shrink only the redundant regions. This is particularly useful when we want to preserve the behaviour of a fine-tuned model on a specific task.

Second, the choice of a genetic algorithm is directly motivated by the nature of the search space. The combination of “number of blocks per layer” and “integer rank per block” creates a discrete and highly irregular landscape where local changes can lead to non-smooth effects on accuracy. Gradient-based methods are not applicable, and naive grid search would be far too expensive. Evolutionary techniques are designed exactly for this kind of configuration problem, where we need to explore many candidates and slowly improve them over time.

Third, the introduction of the BlockLowRankLinear module ensures that the compression is not just theoretical. Many pruning methods look attractive on paper but only yield real gains on specific hardware with sparse kernels. In our case, by changing the architecture so that the model actually stores U , Σ and V instead of W , the reduction in parameter count directly translates into a smaller model on disk and a smaller memory footprint at inference. In other words, the mathematical approximation and the implementation details are aligned.

Overall, combining sensitivity analysis, block-wise SVD and a genetic algorithm gives us a reasonably simple, yet quite expressive, framework to explore adaptive low-rank compression on a real transformer model without relying on gradients or heavy retraining.

Empirical Evaluation:

This section presents the empirical evaluation of our low-rank compression framework driven by a genetic algorithm (GA). The experiments were conducted in two phases. The first phase consists of a single exploration run where the GA is free to mutate both the block configuration (4, 8, or 16 blocks) and the ranks assigned to each block. This run assesses how the algorithm behaves when both structural and parametric compression decisions are simultaneously optimized. The second phase is a controlled comparative study in which three separate experiments are run with a fixed number of blocks (4, 8, or 16), allowing us to isolate the effect of block granularity on accuracy, compression, and convergence dynamics. Finally, we discuss the assumptions, limitations, and trade-offs inherent to our approach.

1. Experimental Setup

The baseline (uncompressed) model was evaluated for accuracy and size to establish reference values. We choose the Compression is performed by applying block-wise SVD approximations to the weight matrices. Each solution is encoded as a chromosome specifying the number of blocks, when applicable, and the SVD rank selected for each block. The GA optimizes a combined objective function balancing accuracy and compression through weighted fitness.

The GA uses a small population, fixed number of generations, elitism, and a moderate mutation rate. In the first exploratory run, mutation is also allowed to change the number of blocks (among 4, 8, 16), enabling dynamic architectural search. In the comparative study, the GA configuration is identical but the number of blocks is fixed, so only rank selection evolves.

For every run, we record the fitness evolution, final rank configuration, final compressed size, and the accuracy of the compressed model.

2. Results

2.1 First Run: Dynamic Block Selection

In the initial exploratory run, the GA simultaneously searched over both the block granularity and the per-block ranks. Despite the larger space, the algorithm converged toward 8- and 16-block configurations more frequently than toward 4 blocks. This behavior reflects strong selective pressure: with mutation allowed to modify the number of blocks, because 4-block solutions are too coarse and often lead to abrupt accuracy drops. The GA tended to stabilize around finer-grained decompositions, which provided more flexible rank allocation and therefore better retention of performance.

The fitness curve shows a consistent improvement during the early generations, after which block configurations stabilize and the remaining improvements arise primarily from adjusting ranks (see Figure a). The first run highlights that the GA naturally gravitates toward higher block counts when structural choices are unrestricted.

2.2 Comparative Study: Fixed Block Configurations (Developed)

After analyzing the exploratory run, we conducted three controlled experiments :one each for 4, 8, and 16 blocks. In these experiments, the number of blocks is fixed, and the GA optimizes only the rank selection. This isolates the effect of block granularity on compression dynamics and solution quality.

4 Blocks :

With coarse partitioning, the chromosome length is short and the search space is correspondingly small. Under the fixed GA budget, convergence is extremely rapid (see Figure b). While this results in high compression ratios, the accuracy tends to suffer (see Figure c). Because each block spans a large portion of the matrix, applying a low-rank approximation removes structural detail on a large scale. The GA's preference for a limited range of ranks reflects the reduced flexibility in distributing compression pressure across layers.

8 Blocks :

At intermediate granularity, the GA benefits from increased flexibility while maintaining a manageable search space. Convergence becomes smoother and more progressive (see Figure b), suggesting that the algorithm is able to explore multiple configurations before settling. The resulting models offer a more favorable balance: compression remains significant, but accuracy degradation is less severe (see Figure c). The rank distributions are more heterogeneous across blocks (see Figure d), indicating that the GA can adapt compression strength to layer sensitivity.

16 Blocks :

The finest-grained decomposition significantly enlarges the chromosome length and search dimensionality. Given the same small population and limited number of generations, convergence becomes slower (see Figure b). Nevertheless, the final solutions tend to preserve accuracy more effectively (see Figure c). Because each block is small, low-rank approximations remove only localized redundant directions, distributing the approximation error across many small components. This produces a notable regularization effect: the compressed model often exhibits smoother weights and increased robustness to overfitting, leading to more stable accuracy relative to the 4-block configurations.

3. Assumptions, Limitations, and Trade-Offs

Our analysis assumes deterministic accuracy evaluation, consistent data splits, and stable implementations of SVD-based approximations. The GA parameters are kept constant across all experiments to ensure comparability. Nevertheless the population size, the number of generations, and the number of layers being compressed are all small, these conditions tend to favor configurations with fewer blocks. With only a limited number of generations and a small population, the GA cannot adequately explore larger genomes, leading to faster, but often premature convergence in low-block settings. Moreover, the small dataset increases the risk of overfitting when running the GA for longer generation counts. Furthermore, each configuration was run only once; conclusions therefore lack statistical uncertainty analysis.

Despite these limitations, several trade-offs emerge clearly. Coarse decompositions enable fast convergence and high compression but risk substantial accuracy loss. Fine decompositions allow more nuanced compression and improved robustness but significantly complicate the search landscape. Finally, allowing the GA to mutate the block count as in the first run, reveals that structural search quickly gravitates toward finer block configurations, reinforcing their functional advantages even before detailed rank optimization is performed.

Conclusion:

Main trends in AI-based LLM optimization

From our review of recent compression methods and the different techniques studied in this report, several patterns clearly emerge:

- **Everything is getting adaptive:**
Most recent methods don't apply a single global compression rate anymore. Bit-widths,

pruning ratios or ranks are chosen *per layer* and sometimes even per head or per channel. GPTQ / AWQ adapt quantization to activation statistics, SVD-LLM V2 adapts rank layer-wise, modern pruning methods adapt sparsity to block importance. The message is: uniform compression is basically over.

- **Compression itself is framed as an AI problem:**
Because many design choices are discrete (which layer to prune, which rank to use, which bit-width, etc.), people increasingly use **search** and **learning**: evolutionary algorithms, reinforcement learning, Bayesian optimization, differentiable NAS, etc. In short, AI is now used to optimize another AI model, instead of doing manual hyper-tuning.
- **Hybrid methods are becoming standard:**
No single technique dominates. The strongest approaches combine several ideas:
 - low-rank + quantization
 - pruning + distillation
 - data selection + KD, etcOur own work sits in this space: low-rank SVD at block level, guided by sensitivity analysis, with a GA exploring the discrete design space.
- **Hardware awareness is slowly entering the loop:**
Many papers now acknowledge that “theoretical” compression (e.g., unstructured sparsity) doesn’t automatically translate into latency gains. There is a growing trend toward hardware-aware search, where the cost function includes real latency or memory usage on a target device.

What we learned from the survey and our implementation

Looking back at the techniques we reviewed, several important observations stood out:

- Quantization is extremely attractive in practice (simple to deploy, big wins on memory), but becomes fragile at very low bit-widths and doesn’t remove operations, only makes them cheaper
- Low-rank methods are more adaptable when it comes to accuracy, but require many design choices (where to factorize, which rank, what to do with residual blocks)
- Pruning is powerful for memory, but its speed benefit depends a lot on the hardware and on the type of sparsity
- Knowledge distillation is the most flexible (you can completely change the architecture), but also the most expensive to run and very dependent on data quality

On the implementation side, a few lessons were more concrete:

- **Layer sensitivity really matters:**
Our noise-based sensitivity analysis (SBLR) was not a cosmetic step. When we tried to compress sensitive layers, accuracy collapsed. Focusing compression on robust layers gave us much better trade-offs.
- **Block-wise low-rank works better than global truncation:**
Decomposing weight matrices into blocks and assigning different ranks to different

blocks captured local structure that a single global SVD would miss. The GA often chose heterogeneous ranks across blocks, which confirms that redundancy is not uniform inside a layer.

- **The search space is huge, but a GA can still find something reasonable.**
Even with a relatively small DistilBERT, the number of possible (rank, blocks, bits) configurations explodes. Our GA only explored a tiny fraction of it, but still converged to stable solutions with several times compression and limited accuracy loss. It also showed how sensitive the search is to the fitness function: ratio-based scores behaved strangely, while a simple linear combination “accuracy – $\beta \times$ size” was easier to interpret and to tune.
- **Architectural changes are mandatory for “real” compression.**
A key practical lesson: just computing SVD and reconstructing back into a standard Linear layer changes nothing in terms of parameter count. The actual gain only appeared once we replaced these layers with a custom BlockLowRankLinear that stores U, Σ and V separately. This step is easy to underestimate when reading papers; implementing it makes the constraint very concrete.

Overall, the combination “sensitivity analysis + block-wise SVD + GA + low-rank layers” gave us a pipeline that is relatively simple, yet already non-trivial compared to naïve global compression.

Future research directions we recommend

Based on both the literature and our experiments, the next steps that make sense to us are:

- Combine low-rank with proper quantization.**
In our project we focused on low-rank structure. Extending the framework to also choose bit-widths (e.g., 8-bit or 4-bit) for the low-rank factors could give multiplicative gains (rank reduction $\times$ quantization). The GA is already well suited to search over these discrete bit choices.
- Move toward hardware-aware fitness.**
We used parameter count as a proxy for “size”. A more realistic objective would include measured latency or energy on the actual target hardware (GPU, CPU, edge device). This would probably change the GA’s preferred solutions, for example favoring fewer blocks if they improve memory access patterns.
- Better evaluation beyond one benchmark.**
We only looked at SST-2 accuracy. A more robust study would test:
 - generalization to other tasks
 - robustness to distribution shifts
 - and preservation of higher-level behaviours (e.g., reasoning, calibration)Compression can interact with these properties in non-obvious ways.

In short, the project convinced us that LLM compression is not just about applying one clever trick, but about navigating a multi-objective design space. Our small contribution is one example of how AI techniques (here, a genetic algorithm) can help explore that space in a more systematic way.

Appendix:

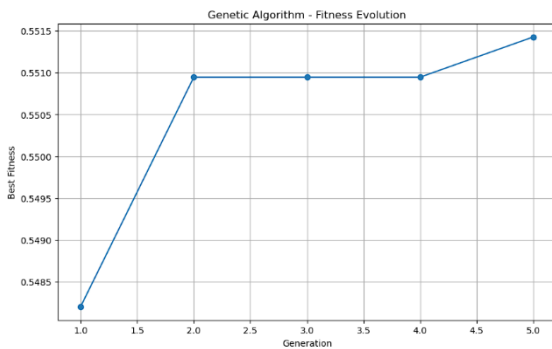


Figure a: Fitness Evolution

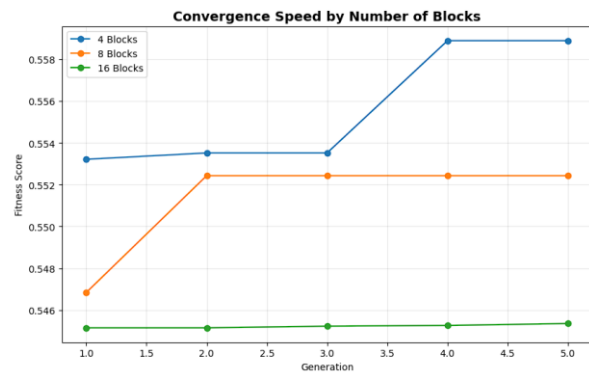


Figure b: Fitness evolution by Number of Blocks

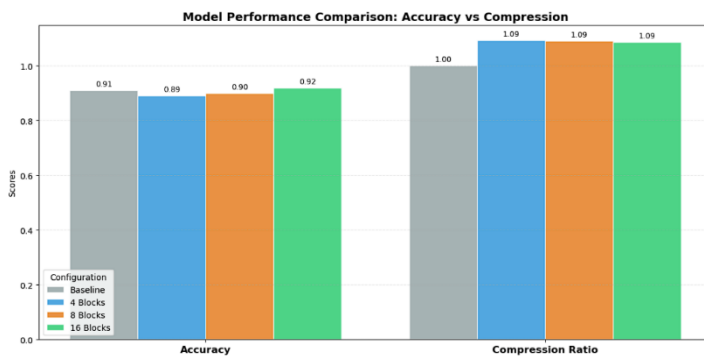


Figure c: Model Performance by Number of Blocks

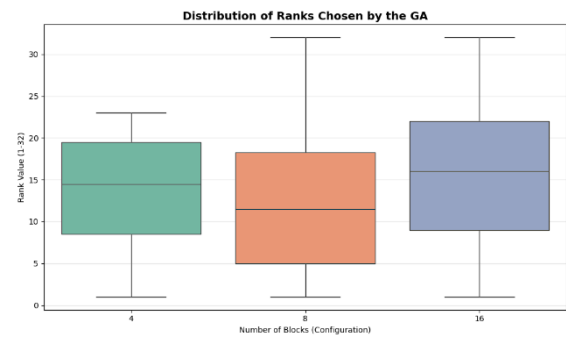


Figure d: Model Performance Comparison

Resources:

[Optimizing LLMs for Resource-Constrained Environments: A Survey of Model Compression Techniques | IEEE Conference Publication | IEEE Xplore](#)

[Designing Large Foundation Models for Efficient Training and Inference: A Survey - Astrophysics Data System](#)

[\[2503.12340\] SVD-LLM V2: Optimizing Singular Value Truncation for Large Language Model Compression](#)

Runtime Adaptive Pruning for LLM Inference: <https://arxiv.org/pdf/2505.17138>

Truncation-aware SVD for LLM compression: <https://arxiv.org/pdf/2403.07378>